Steven R. Dunbar
Department of Mathematics
203 Avery Hall
University of Nebraska-Lincoln
Lincoln, NE 68588-0130
http://www.math.unl.edu
Voice: 402-472-3731
Fax: 402-472-8466

Topics in Probability Theory and Stochastic Processes
Steven R. Dunbar

Games as Markov Chains

Rating

Mathematically Mature: may contain mathematics beyond calculus with proofs.

Section Starter Question

What feature of a board game is necessary for it to be modeled as a Markov chain? Can you think of a board game that can be modeled as a Markov chain?

Key Concepts

1. *Tenzi* is a simple dice game for multiple players. The gameplay of Tenzi for a single player can be modeled with a Markov chain with 10 transient states and 1 absorbing state. Markov chain methods give the distribution of the number of rolls necessary to complete the game, along with the mean and variance.

2. The childrens' game *Chutes and Ladders* is a 100 state Markov chain with a final absorbing state and Markov chain methods give the expected playing time.

3. *Count Your Chickens!* is a cooperative counting game aimed at preschoolers. Absorbing Markov chains give the probability of winning a game of *Count Your Chickens!* and the expected number of chicks at the end of the game. The main interest is classifying and counting the states, leading to a 165×165 transition probability matrix.

4. *Risk* is a board game of battles between two players, and each battle consists of one or more rounds of combat between A attacking armies and D defending armies. The goal is to develop the probability of winning the battle as a function of A and D using a Markov chain with $AD + A + D$ states.

Vocabulary

1. *Tenzi* is a simple dice game for multiple players.
2. The childrens' game *Chutes and Ladders* is a board game that can be modeled as a 100 state Markov chain with a final absorbing state.
3. *Count Your Chickens!* is a cooperative counting board game aimed at preschoolers.
4. *Risk* is a board game of battles between two players, and each battle consists of one or more rounds of combat between A attacking armies and D defending armies.

Mathematical Ideas

Tenzi

Tenzi is a simple dice game for multiple players. Each player has 10 six-sided fair dice in their hands. Taking turns, one players yells, e.g., "Tenzi Four!" Everyone starts rolling their dice as quickly as possible. Whenever a "four" comes up, the player sets it aside and rolls the remaining dice. Eventually the player achieves all 10 "fours" and yells, "Tenzi!".

A Markov chain models the gameplay of *Tenzi* for a single player. Each state in the Markov chain represents the player's accumulated number of dice matching the target number. So at any point in the game, a player will be in one of the 11 states $\{0, 1, 2, 3, \ldots, 10\}$. A player in state 0 has not yet rolled any dice matching the target number, and a player in state 10 will yell "Tenzi!' Each time a player rolls the dice, the player has a chance to move up to a higher-numbered state.

Each entry in the transition probability matrix is a binomial probability. A player in state i rolls the remaining $10 - i$ dice. The player moves up to state $j > i$ if exactly $j - i$ of the dice match the target number. So, the

probability of transitioning from state i to state j in one roll of the dice is $\binom{10-i}{j-i}\left(\frac{1}{6}\right)^{j-i}\left(\frac{5}{6}\right)10-j$ for $0 \leq i,\ j \leq 10$ and $i \leq j$. A player will never go backwards to a previous state in the game, so the probability is 0 for $i > j$. Thus the states are not communicating, the Markov Chain is not recurrent, states 0 through 9 are transient and the state 10 is an absorbing state.

The full 11×11 transition probability matrix P is approximately

$$\begin{pmatrix}
0.162 & 0.323 & 0.291 & 0.155 & 0.0543 & 0.013 & 0.00217 & 2.48 \cdot 10^{-4} & 1.86 \cdot 10^{-5} & 8.27 \cdot 10^{-7} & 1.65 \cdot 10^{-8} \\
0.0 & 0.194 & 0.349 & 0.279 & 0.13 & 0.0391 & 0.00781 & 0.00104 & 8.93 \cdot 10^{-5} & 4.47 \cdot 10^{-6} & 9.92 \cdot 10^{-8} \\
0.0 & 0.0 & 0.233 & 0.372 & 0.26 & 0.104 & 0.026 & 0.00417 & 4.17 \cdot 10^{-4} & 2.38 \cdot 10^{-5} & 5.95 \cdot 10^{-7} \\
0.0 & 0.0 & 0.0 & 0.279 & 0.391 & 0.234 & 0.0781 & 0.0156 & 0.00188 & 1.25 \cdot 10^{-4} & 3.57 \cdot 10^{-6} \\
0.0 & 0.0 & 0.0 & 0.0 & 0.335 & 0.402 & 0.201 & 0.0536 & 0.00804 & 6.43 \cdot 10^{-4} & 2.14 \cdot 10^{-5} \\
0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.402 & 0.402 & 0.161 & 0.0322 & 0.00322 & 1.29 \cdot 10^{-4} \\
0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.482 & 0.386 & 0.116 & 0.0154 & 7.72 \cdot 10^{-4} \\
0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.579 & 0.347 & 0.0694 & 0.00463 \\
0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.694 & 0.278 & 0.0278 \\
0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.833 & 0.167 \\
0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 1.0
\end{pmatrix}.$$

The initial distribution is $X_0 = (1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)$. Since the probabilities are accumulating in state 10, the entry in the last entry of X_k is the probability of getting Tenzi in k or fewer rolls. For example, the entry in the last entry of X_{20} is 0.7677. The player has a 76.77% chance the game is finished by the 20th roll. The player may have finished on the 20th roll, or possibly finished on some previous roll and is waiting in state 10. The resulting cumulative probability function $P(k)$ for a player getting Tenzi on or before the kth roll is in Figure 1. To find $p(k)$, the probability of getting Tenzi in exactly k rolls, subtract the probability that the player already had Tenzi by the $k-1$ roll. This gives the probability density function $p(k) = P(k) - P(k-1)$ also shown in Figure 1. The maximum of this probability function is at $k = 13$; a player has a 0.0705 probability of getting Tenzi on their 13th roll. Using these probabilities, the mean of the distribution is $\sum_{k=1}^{\infty} kp(k)$. Approximate this sum through $k = 50$ as 16.5. This is the average number of rolls needed for a player to get Tenzi.

The fundamental matrix provides another way of finding the mean. The transition probability matrix is already partitioned as

$$P = \begin{pmatrix} Q & R \\ 0 & I \end{pmatrix}$$

where Q is the 10×10 matrix containing the transition probabilities among the transient states, R is a 10×1 column matrix containing the probabilities of going from each of the non-absorbing states to state 10 in one roll, 0 is a 1×10 zero matrix, and I is the 1×1 identity matrix representing the

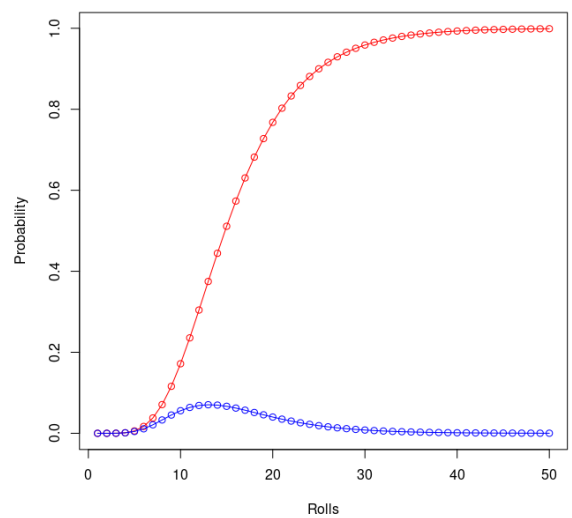

Figure 1: Probability density function (in blue) and cumulative distribution function (in red) of waiting time to win in Tenzi.

absorbing state 10. Then N is approximately

$$\begin{pmatrix} 1.193 & 0.478 & 0.669 & 0.787 & 0.915 & 1.097 & 1.371 & 1.828 & 2.742 & 5.485 \\ 0.000 & 1.240 & 0.564 & 0.771 & 0.917 & 1.097 & 1.371 & 1.828 & 2.742 & 5.485 \\ 0.000 & 0.000 & 1.303 & 0.673 & 0.905 & 1.099 & 1.371 & 1.828 & 2.742 & 5.485 \\ 0.000 & 0.000 & 0.000 & 1.387 & 0.815 & 1.091 & 1.373 & 1.828 & 2.742 & 5.485 \\ 0.000 & 0.000 & 0.000 & 0.000 & 1.504 & 1.010 & 1.368 & 1.829 & 2.742 & 5.485 \\ 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 1.672 & 1.298 & 1.826 & 2.743 & 5.485 \\ 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 1.931 & 1.769 & 2.742 & 5.485 \\ 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 2.374 & 2.697 & 5.485 \\ 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 3.273 & 5.455 \\ 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 0.000 & 6.000 \end{pmatrix}.$$

The theorems from Waiting Time to Absorption give a lot of information about the progress of the game. The entry N_{ij} (counting i and j from 0) represents the expected number of turns spent in state j given that the player started in state i. Notice that the diagonal entries are all greater than 1. If a player starts the game in state i then at least one roll, the first roll, is in state i. Entries in later states $j > i$ can be less than 1 because it is possible for a

0	1	2	3	4	5	6	7	8	9
1.19	0.48	0.67	0.79	0.92	1.10	1.37	1.83	2.74	5.48

Table 1: Expected number of times the chain will be in each state.

0	1	2	3	4	5	6	7	8	9
6.83	6.81	6.78	6.75	6.70	6.64	6.55	6.41	6.14	5.48

Table 2: Standard deviation of the number of times the chain will be in each state.

player to skip a state entirely. Notice the 6 in the bottom right corner gives the average number of rolls spent in state 9 given that the player started in state 9. This makes sense; in state 9, the player has just one die left to roll. It will take 6 rolls on average to roll the given target number, this is the mean of a geometric random variable with $p = 1/6$. Since players start the game of *Tenzi* in state 0 the first row of matrix N gives the expected number of rolls a player will spend in each state, see Table 1. This makes sense, for state 0, there is always the first roll, but usually the player moves to another state. For states 1 through 4, it is common to get more than one of the target number, so those states often get skipped over. For states 5 to 9, it takes increasingly longer to collect the target number.

The sum of the first row is the mean number of turns to go from state 0 to the absorbing state 10, specifically, 16.564849, agreeing with the approximate mean from the pdf. The standard deviation of the number of turns the player will spend in state i before reaching the absorbing state 10 from state 0 is in Table 2. This also makes sense; in state 9, then you have just one die left to roll. The entry 5.48 is the standard deviation of a geometric random variable with $p = 1/6$.

Chutes and Ladders

The childrens' game *Chutes and Ladders* (known in the United Kingdom as *Snakes and Ladders*) derives from an old Hindu game *Leela* teaching moral lessons. The game described here is based on the U.S. version. The game is a 100 state absorbing Markov chain and the goal is to investigate the expected playing time.

The game is for two or more players. The game is played on a 10 by 10 board with squares numbered 1 to 100 that are the states of the game.

Rather than add an additional state 0 as the starting position, instead make an initial uniform distribution of starting states, see below for details. (The scripts in the R statistical computing language which indexes vectors from 1, in contrast to say, Python, which indexes from 0 motivates this choice of 100 states with an initial distribution. With 101 states including a starting state, the state indices would be one greater than the spaces marked on the board, leading to potential confusion.) Since the goal is to find the expected playing time, remember to add 1 to the playing time for the initial move to be in one of the starting states. Players roll a fair six-sided die (or use a fair six-section spinner) to determine how many squares to advance their token on each turn. Two or more players may occupy the same square simultaneously. The first player to reach square 100 exactly wins. In the final 5 states, if a move would take a player beyond the last square, the player does not move. The game board is also marked with chutes and ladders connecting various squares on the board. The special feature of the game is that if a token lands on a square at the bottom of a ladder the token immediately moves to the top of the ladder. Similarly if the token lands on a square at the top of a chute, it immediately moves to the bottom of the chute. The game has 10 chutes and 9 ladders with beginning and end states given by the following ordered pairs

- Ladders: $(1, 38)$, $(4, 14)$, $(9, 31)$, $(21, 42)$, $(28, 84)$, $(36, 44)$, $(51, 67)$, $(71, 91)$, $(80, 100)$;
- Chutes: $(16, 6)$, $(47, 26)$, $(49, 11)$, $(56, 53)$, $(62, 19)$, $(64, 60)$, $(87, 24)$, $(93, 73)$, $(95, 75)$, $(98, 78)$.

This means that 19 states, the beginnings of the chutes and ladders, are inaccessible states. For example, a player is never actually in state 1, for if the token lands on square 1, the ladder $(1, 38)$ immediately deposits the player into state 38. Although the game is a race among two or more players, the players move independently, so it is sufficient to consider just one player moving and ask "How long does it typically take a single player to finish a game by reaching square 100?"

Although the game has 100 squares, it is possible to model the game as a Markov chain with fewer states, see [1]. Adding a starting state 0, but deleting each inaccessible state corresponding to a chute or a ladder leaves 82 states. The probability transition matrix is an 82×82 matrix with most rows containing 6 consecutive probabilities of $1/6$ each. There are two kinds

of exceptions. A row corresponding to any state reachable from a chute or ladder must account for the long-distance transitions by the chute or ladder. The last 5 rows show the fact that a player must reach state 100 exactly, a die roll causing the player to go past state 100 results in no move.

Here the model uses a 100-state Markov chain with a 100×100 transition probability matrix derived from matrices analogous to those in the Monopoly model. First use a 100×100 circulant (almost, excepting the last 5 rows) dice roll matrix R () followed by sparse 100×100 chutes and ladders transition matrices C and L. The matrix L is just the identity matrix but, for instance the 1 in row 1 and column 1, moves to row 1 and column 38 to indicate ladder $(1, 38)$. Likewise for the 18 other ladders and chutes. The result RLC is a 100×100 transition probability matrix with transient states 1 through 99 and absorbing state 100. The game starts in states 38, 2, 3, 14, 5 and 6, each with probability $\frac{1}{6}$. The expected waiting time to absorption is the average over the expected waiting time to absorption from each state, $\frac{1}{6}\left(W(38) + W(2) + W(3) + W(14) + W(5) + W(6)\right) = 38.22512$. Adding for the initial move to the starting states gives 39.22512, agreeing with [1]. Using Theorem 3 from Waiting Time to Absorption the standard deviation of the waiting time to absorption is approximately 25.2 turns. As a final note, observe that the waiting time to absorption from state 99 is 6 with standard deviation $\sqrt{30} \approx 5.48$, exactly as expected for a geometric random variable with $p = 1/6$, waiting to roll a 1 to enter the last state.

In [1] the authors investigate the sensitivity of the waiting time to adding or removing chutes and ladders. For example, the expected waiting time to absorption for a game with no chutes and ladders is 34.68571 moves. See the exercises for some examples.

Count Your Chickens

Count Your Chickens!, produced by Peaceable Kingdom, is a simple game for preschoolers. In this cooperative game, play begins with forty chicks spread throughout the board and a mama chicken at the start square. The end square is the mama chicken's coop, and the mama chicken moves progressively closer to this square as the game unfolds. The object of the game is to have all 40 chicks in the chicken coop when the mama chicken arrives there. The game board has 41 squares numbered in order, with 30 squares containing one of five characters: a cow, a sheep, a pig, a dog, or a tractor. Five squares are blue. The final square, the chicken coop, contains all 5 farm

characters. Players take turns spinning a spinner with 6 segments of equal area. Five segments match the farm characters from the board and a sixth segment has a fox character. When a player spins the spinner, one of two things can happen. If the player spins one of the five characters drawn on the board, she moves the mama chicken token along the path toward the coop to the next instance of the character. The player counts the squares moved, and adds this number of chicks to the chicken coop. If the square landed on is blue, the player adds one additional chick to the coop. There are only a total of 40 chicks on the board though, so if the mama chicken moves two squares but there are already 39 chicks in the coop, only one chick gets added. If a fox is spun, the player must remove one chick from the coop and place it back on the board. If there are no chicks in the coop then spinning a fox has no effect.

For example, suppose the mama chicken is at the start square. If the player spins a sheep, then the mama chicken moves to square 2 (the first sheep) and places two chicks in the coop. If the player next spins a tractor then the mama chicken moves 2 squares to square 4 and places 3 more chicks in the coop because square 4 is blue. If the first spin is a fox then nothing happens because at the start of the game there are no chicks in the coop. The game continues in this manner until the mama chicken reaches the coop. The players win if all 40 chicks are in the coop when the mama chicken arrives there; otherwise the players lose, this is a cooperative game.

The goal here is to answer the following questions:

1. What is the probability of winning a game of *Count Your Chickens!*? That is, what is the probability of landing on square 40 with 40 chicks in the coop?

2. What is the expected number of chicks in the coop at the end of the game?

3. What effect do the blue squares have on the probability of winning the game? For example, the original game has five particular squares colored blue; are there other choices of five blue squares that increase the probability of winning the game?

The mathematical interest here is creating the state space, leading to a large transition probability matrix. A state in *Count Your Chickens!* is an ordered pair (a, b) where a is the mama chicken's position on the board and

0 Start	1	2 Sheep	3 Cow	4	5 Cow	6	7 Sheep	8 Coop Sheep Cow

Figure 2: Smaller game board for the example *Count Your Chickens!*

b is the number of chicks in the coop. For win probabilities only, to minimize the number of states and cut down on computation, include a "loss" state and a "win" state. The idea is that while $(39, 35)$, for example, is a possible state of the game, it is impossible to win from such a configuration. All such ordered pairs from which it is impossible to win are grouped together as a loss state. The win state is $(40, 40)$, the mama chicken must reach the last square of the board with all of the chicks in the coop.

The following example illustrates the construction of such a chain with a smaller board, see Figure 2.

Example. This smaller game uses only two farm characters – a sheep and a cow. There are 8 squares on the board after the start square and 8 chicks to return to the coop. The spinner has three equal-sized segments illustrated with a cow, a sheep, and a fox. Number only the squares with a character, since it is not possible to land on a blank square.

A state of the game is an ordered pair (i, j) where i is the position of the mama chicken and j is the number of chicks in the coop. Identify some pairs with the loss state because if j is small enough relative to the mama chicken's position, it will be impossible for all 8 chicks to be in the coop at the end of the game. The state $(2, 0)$ is such an example: if the mama chicken is on square 2 and there are no chicks in the coop, the game will end with at most 7 chicks in the coop. Let b_i^R be the number of blue squares after square i on the path and b_i^L the number of blue squares that come before square i, including square i. Note $b_i^R + b_i^L$ is the total number of blue squares in the game. Then for square i, the states are (i, j) with $\max(i - b_i^R, 0) \leq j \leq \min(i + b_i^L, 8)$. For example, for square 3, $b_3^R = 1$ and $b_3^L = 1$ since there is one blue square to the right of square 3 and one blue square to the left of square 3. Thus square 3 corresponds to possible states $(3, 2)$, $(3, 3)$, $(3, 4)$. It is not possible to have more than 4 chicks in the coop on square 3, and if the coop has 0 or

1 chicks on square 3 it is impossible to win the game. All the states are

$$\{(0,0),(2,1),(2,2),(2,3),(3,2),(3,3),\\ (3,4),(5,5),(5,6),(5,7),(7,7),(7,8),\text{win},\text{loss}\}$$

for this board. For the purposes of building a transition matrix think of $(0,0)$ as state 1, $(2,1)$ as state 2, $(2,2)$ as state 3, ..., and "loss" as state 14. Entry $p_{i,j}$ in the transition matrix for the game is the probability of moving from state i to state j. For example, in row 1 of the transition matrix, start in position $(0,0)$ on the game board. There is a $1/3$ probability of rolling a fox; this spin keeps the token in position $(0,0)$, so $p_{1,1} = 1/3$. Spinning a sheep is the next possibility, putting the token in position $(2,3)$, or a cow, putting the token in position $(3,3)$. This gives $p_{1,4} = 1/3$ and $p_{1,6} = 1/3$. All other entries in row 1 are 0. Starting in state $(2,1)$, then spinning the sheep puts the token in position $(7,6)$, or the loss state, because if there are only 6 chicks in the coop upon reaching square 7, it is impossible to win. Spinning the fox also puts the game in the loss state, so entry $p_{2,14} = 2/3$. Spinning the cow puts the game in state $(3,2)$, so $p_{2,5} = 1/3$. All other entries in row two are 0. Continuing in this way, the transition matrix for this game is

$$\begin{pmatrix}
\frac{1}{3} & 0 & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{2}{3} \\
0 & \frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & 0 \\
0 & 0 & \frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & 0 & 0 & 0 & \frac{2}{3} \\
0 & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & \frac{1}{3} & \frac{1}{3} \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} & \frac{1}{3} & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & \frac{1}{3} & \frac{1}{3} & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{2}{3} & \frac{1}{3} \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & \frac{2}{3} & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1
\end{pmatrix}.$$

The state and transition probability matrix models the small game as an absorbing Markov chain. The loss and win states are both absorbing since each of these states transitions to itself with probability 1, and it is possible

to reach one of these two states from any other state. The transition matrix is in the canonical form

$$\begin{pmatrix} Q & R \\ 0 & I \end{pmatrix}$$

where I is the identity matrix. With the transition matrix in canonical form, the fundamental matrix is

$$N = (I-Q)^{-1} = \begin{pmatrix}
\frac{3}{2} & \frac{1}{18} & \frac{1}{6} & \frac{1}{2} & \frac{2}{9} & \frac{11}{18} & \frac{1}{6} & \frac{4}{27} & \frac{2}{9} & \frac{1}{18} & \frac{67}{162} & \frac{17}{54} \\
0 & 1 & 0 & 0 & \frac{1}{3} & 0 & 0 & \frac{1}{9} & 0 & 0 & \frac{1}{27} & 0 \\
0 & \frac{1}{3} & 1 & 0 & \frac{2}{9} & \frac{1}{3} & 0 & \frac{1}{9} & \frac{1}{9} & 0 & \frac{40}{81} & \frac{1}{27} \\
0 & \frac{1}{9} & \frac{1}{3} & 1 & \frac{1}{9} & \frac{2}{9} & \frac{1}{3} & \frac{2}{27} & \frac{1}{9} & \frac{1}{9} & \frac{31}{81} & \frac{14}{27} \\
0 & 0 & 0 & 0 & 1 & 0 & 0 & \frac{1}{3} & 0 & 0 & \frac{1}{9} & 0 \\
0 & 0 & 0 & 0 & \frac{1}{3} & 1 & 0 & \frac{2}{9} & \frac{1}{3} & 0 & \frac{4}{9} & \frac{1}{9} \\
0 & 0 & 0 & 0 & \frac{1}{9} & \frac{1}{3} & 1 & \frac{1}{9} & \frac{2}{9} & \frac{1}{3} & \frac{26}{81} & \frac{14}{27} \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & \frac{1}{3} & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & 1 & 0 & \frac{2}{9} & \frac{1}{3} \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{9} & \frac{1}{3} & 1 & \frac{5}{27} & \frac{4}{9} \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & 1
\end{pmatrix}.$$

Let $B = N \times R$. If the game is in the ith non-absorbing state then the probability that the game terminates by reaching the jth absorbing state is b_{ij}. Then

$$B = \begin{pmatrix}
\frac{305}{486} & \frac{181}{486} \\
\frac{5}{81} & \frac{76}{81} \\
\frac{104}{243} & \frac{139}{243} \\
\frac{170}{243} & \frac{73}{243} \\
\frac{5}{27} & \frac{22}{27} \\
\frac{5}{9} & \frac{4}{9} \\
\frac{190}{243} & \frac{53}{243} \\
\frac{5}{9} & \frac{4}{9} \\
\frac{22}{27} & \frac{5}{27} \\
\frac{73}{81} & \frac{8}{81} \\
\frac{2}{3} & \frac{1}{3} \\
\frac{8}{9} & \frac{1}{9}
\end{pmatrix}.$$

From the first row of B, this game has a win probability of $305/486 \approx 0.6276$ and a loss probability of $181/486 \approx 0.3724$ when a player begins from the start square. One of the nice aspects of using a Markov chain for these

calculations is that a player can update the win probability moving through the game. For example, suppose the player starts by spinning a sheep and then spins a fox. This state of the game corresponds to the state $(2, 1)$, which corresponds to the second row of B. Therefore in such a situation the player would know that her win probability is now only $5/81 \approx 0.0617$.

The computations for the general game are similar except that the matrices Q and R are much larger. Q is 163×163 and R is 163×2 and therefore B is 163×2. It is impractical to enter and compute with such large matrices by hand. The Scripts section has an R program to build the matrices and compute the win probability. The matrix Q in the full game looks similar to the matrix Q in the small example: the matrix is mostly zeroes punctuated sparsely with small diagonals of entries of value $1/6$, instead of $1/3$. The top left entry of B in this case is 0.6410, and the probability of winning a game of *Count Your Chickens!* is 0.641.

Risk

The board game *Risk* game has a world map separated into 42 territories and two or more players. Each player has a number of tokens used as armies. Each token represents one army. A player may use some or all but one of the armies to attack the armies in an adjacent opponent-controlled territory. The goal of a player is either to conquer the world through battles or to fulfill a task assigned at the beginning of the game. This task can be destroying a specific enemy or conquering a specific area.

The game consists of battles between two players, and each battle consists of one or more rounds of combat. This section uses Markov chains to analyze the stochastic progress of a battle between two players in the board game *Risk*. Standard Markov chain methods give the probabilities of victory and additionally the expected losses in battle. At each turn, a player must decide whether or not to attack a territory. The first question is the following: *If you attack a territory with your armies, what is the probability that you will capture this territory?* Of course, the probability that you will capture a territory could be high while the expected loss is also high. Therefore, the second question is the following: *If you engage in a war, how many or your armies should you expect to remain, depending on the number of armies your opponent has on that territory?*

At each turn, a player occupying some region may attack a neighboring territory. An attacker must leave at least one army on the occupied territory,

	Number of Armies		Number of Dice	
Case	Attacker	Defender	Attacker	Defender
I	1	1	1	1
II	2	1	2	1
III	≥ 3	1	3	1
IV	1	≥ 2	1	2
V	1	≥ 2	1	2
VI	≥ 3	≥ 2	1	2

Table 3: The number of dice each player rolls according to the number of attacker and defender armies.

and use at least one army in an attack. Thus a player who has at least two armies (one to keep on the territory and one to attack a territory) may attack a neighbor territory. A player who decides to attack a territory declares war on the opponent and the two players engage in a sequence of combats with random outcomes determined by dice. Outcomes of the defender's and the attacker's dice determine each battle and the ultimate outcome of the war. An attacker may withdraw before the war ends. An attacker who does not withdraw either destroys all the defender armies in that territory and occupies that territory, or loses all the attacking armies and fails to conquer the territory.

The number of armies an attacker or a defender possesses at the start of a each combat determines the number of dice each player rolls. To begin a round of battle, the attacker will roll one die if attacking with a single army, two dice if the attacker has two armies present, and three dice if the attacking player has three or more armies involved in the battle. A defender rolls one die with one army and two dice with two or more armies in the embattled territory. According to the number of dice each opponent rolls, there are six different cases, given in Table 3.

At each turn, after rolling the dice, each side sets the dice in descending order and then pairs off the two sets of dice, matching the highest attacking die roll against the highest defending die roll. If each player is using at least two dice, then the second highest die rolls are similarly matched. The attacker loses one army for each die that is less than or equal to the corresponding defender's die. The defender loses one army for each die that is strictly less than the corresponding attacker's die. Note that ties favor the defender, so each tie results in the loss of an attacking army. After tak-

Roll	Armies		Dice Rolled		Dice Outcome		Losses	
	Att	Def	Att	Def	Att	Def	Att	Def
1	4	3	3	2	5,4,3	6,3	1	1
2	3	2	3	2	5,5,3	5,5	2	0
3	1	2	1	2	6	4,3	0	1
4	1	1	1	1	5	6	1	0
5	0	1						

Table 4: An example of a battle in *Risk*

.

ing away the armies lost at that turn, the players roll the dice again. This sequence continues until one side loses all of its armies. Table 4 shows the progress of a typical war for a country. This war took four rounds. At the end of the fourth turn, the attacker lost all of the attacking armies and the defender wins the war. This table also serves as a reminder of the number of dice rolled in various situations, never more than three for the attacker, and never more than two for the defender.

It may be disadvantageous to take control of a new territory by winning a war with only one army remaining, emphasizing the importance of the second question: *If you engage in a war, how many armies should you expect to lose depending on the number of armies your opponent has on that territory?*

Let A be the number of attacking armies and D the number of defending armies. The values of A and D characterize the state of the system at a given time. Let X_n be the state of the system at the beginning of the nth turn:

$$X_n = (a_n, d_n), 0 \leq a_n \leq A, 0 \leq d_n \leq D$$

where a_n and d_n are the number of attacking and defending armies respectively. The initial state of the system is $X_0 = (A, D)$. If one side loses all its armies then that side loses the war; i.e., if $X_n = (0, d_n)$ with $d_n > 0$, then the attacker has lost the war at the end of the turn. Similarly, if $X_n = (a_n, 0)$, with $a_n > 0$, then the attacker has won the war at the end of the turn. If we know the number of armies each side has at a given round, then we can calculate the probability that each side wins or loses the war without knowing the states of the system prior to that round. In other words, the battle process has the Markov property.

The AD states where both a and d are positive are transient. The $A + D$ states where either $a = 0$ or $d = 0$ are absorbing. Order the $AD + (A + D)$

possible states so that the AD transient states precede the $A+D$ absorbing states. Order the transient states with the attacker armies decreasing along a row for constant defenders, and the defender armies decreasing row by row, see Figure 3. The transient states are, in order

$$\begin{array}{ll} (A,D),(A-1,D),(A-2,D),\ldots,(1,D), & \\ (A,D-1),(A-1,D-1),\ldots,(1,D-1), & \\ \ldots & (A,1),(A-1,1),\ldots,(1,1) \end{array}$$

and order the absorbing states by decreasing desirability of outcome for the attacker, including the absorbing states where the defender wins

$$(A,0),(A-1,0),\ldots,(1,0),(0,1),(0,2),\ldots,(0,D).$$

Note that this order is the reverse of the state order in [4]. The order here proceeds in the general direction of the battle, with the first state for the initial numbers of armies, then decreasing number of armies for both players. See Figure 3 for a diagram of the possible state transitions when $A = 5$ and $D = 4$. This figure is key for understanding the Markov chain. The small number below each ordered pair state is the index of the state for the transition probability matrix. Similar to European language reading directions, the state indices increase from left to right and top to bottom, so the state of the system flows from the upper left corner to the bottom or right margin. The diagram divides into 6 regions, 3 for the attacker, and 2 for the defender depending on the number of dice each is rolling in a battle, as already outlined in Table 3. For example, the lower right region corresponds to Case I in Table 3 and the transition probabilities π_{111} and π_{110} detailed below in Table 5. The largest region in the upper left corresponds to Case VI in Table 3 and the transition probabilities π_{32k} in Table 5. Likewise for the other regions. Figure 3 labels the possible state transitions with arrows and colors corresponding to the transition probabilities in Table 5.

The states and the coordinates of states on the diagram can be confusing. For instance, the initial armies state $(5,4)$ is located at location $(1,4)$ on the diagram. As the attacking armies decrease, the first coordinate of the location of the state increases. As the defending armies decrease, the second coordinate of the location decreases. This means first that attention must be paid to programming the chart and second to not confuse location with state.

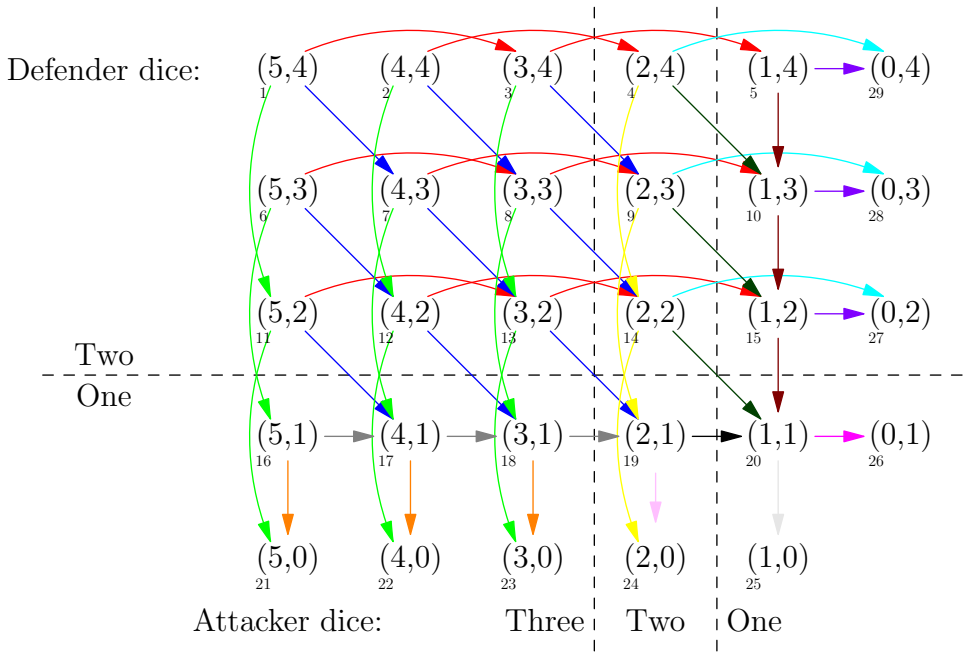

Figure 3: The state space and transitions of the Markov chain for a war in *Risk*. Transitions of the same color have the same probability.

Under this ordering, the transition probability matrix takes the block-matrix form

$$P = \begin{pmatrix} Q & R \\ 0 & I \end{pmatrix}$$

where the $(AD) \times (AD)$ matrix Q contains the probabilities of going from one transient state to another, and the $(AD) \times (A + D)$ matrix R contains the probabilities of going from a transient state into an absorbing state. P contains only 14 distinct probabilities, based on the number of dice rolled and the armies lost as a result. Let π_{ijk} denote the probability that the defender loses k armies when rolling j dice against an attacker rolling i dice, as given in Table 5. To obtain the π_{ijk}, determine the joint probabilities associated with the best and second best roll from 2 or 3 six-sided fair dice. Let Y_1, Y_2, Y_3 denote the unordered outcome for an attacker when rolling three dice and let W_1, W_2 denote the unordered outcome for an attacker when rolling two dice. Let Z_1, Z_2 denote the unordered outcome for a defender rolling two dice. Then Y_1, Y_2, Y_3, W_1, W_2 and Z_1, Z_2 are random samples from the discrete uniform distribution on the integers 1 through 6:

$$\mathbb{P}\left[Y_j = y\right] = \begin{cases} \frac{1}{6} & y = 1, 2, 3, 4, 5, 6 \\ 0 & \text{else.} \end{cases}$$

Taking order into account, denoted using superscripts, as in $Y^{(1)} \geq Y^{(2)} \geq Y^{(3)}$, the ordered random variables are the *order statistics*. The joint distributions of the order statistics for specification of π_{ijk} use straightforward techniques of enumeration. When rolling two dice, the joint distribution of $(Z^{(1)}, Z^{(2)})$ is

$$\mathbb{P}\left[Z^{(1)} = z^{(1)}, Z^{(2)} = z^{(2)}\right] = \begin{cases} \frac{1}{36} & z^{(1)} = z^{(2)} \\ \frac{2}{36} & z^{(1)} > z^{(2)} \\ 0 & \text{else.} \end{cases}$$

See Figure 4 for a visualization of how to obtain this probability for $Z^{(1)} = 5, Z^{(2)} = 3$. The *marginal distribution* of the best roll $Z^{(1)}$ sums the joint distribution over values of $z^{(2)}$:

$$\mathbb{P}\left[Z^{(1)} = z^{(1)}\right] = \frac{2z^{(1)} - 1}{36}$$

for $z^{(1)} = 1, 2, 3, 4, 5, 6$. See Figure 5 for a visualization of how to obtain this probability for $Z^{(1)} = 5$. When rolling three dice, the distribution of the best

	1	2	3	4	5	6
1						
2						
3					$\frac{1}{36}$	
4						
5			$\frac{1}{36}$			
6						

Figure 4: Visualization of $Z^{(1)} = 5$, $Z^{(2)} = 3$.

	1	2	3	4	5	6
1	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$
2	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$
3	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$
4	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$
5	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$
6	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$	$\frac{1}{36}$

Figure 5: Visualization of $Z^{(1)} = 5$.

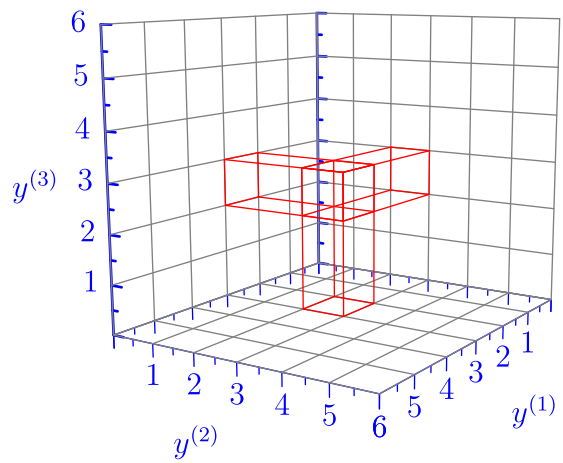

Figure 6: Visualization of $Y^{(1)} = 3, Y^{(2)} = 3$.

two rolls is

$$\mathbb{P}\left[Y^{(1)} = y^{(1)}, Y^{(2)} = y^{(2)}\right] = \begin{cases} \frac{3y^{(1)}-2}{216} & y^{(1)} = y^{(2)} \\ \frac{6y^{(2)}}{36} & y^{(1)} > y^{(2)} \\ 0 & \text{else.} \end{cases}$$

See Figure 6 for a visualization of how to obtain this probability for $Y^{(1)} = 3$, $Y^{(2)} = 3$. and the marginal distribution of the best roll is

$$\mathbb{P}\left[Y^{(1)} = y^{(1)}\right] = \frac{1 - 3y^{(1)} + 3\left(y^{(1)}\right)^2}{216}$$

for $y^{(1)} = 1, 2, 3, 4, 5, 6$. All of the probabilities are 0 for arguments that are not positive integers less than or equal to 6. The joint distribution of $W^{(1)}$ and $W^{(2)}$ is the same as that for $Z^{(1)}$ and $Z^{(2)}$.

Now it is possible to calculate specific transition probabilities using these

marginal distributions. For example

$$\begin{aligned}
\pi_{322} &= \mathbb{P}\left[Y^{(1)} > Z^{(1)}, Y^{(2)} > Z^{(2)}\right] \\
&= \sum_{z_1}^{5} \sum_{z_2=1}^{z_1} \mathbb{P}\left[Y^{(1)} > z_1, Y^{(2)} > z_2\right] \mathbb{P}\left[Z^{(1)} = z_1, Z^{(2)} = z_2\right] \\
&= \sum_{z_1=1}^{5} \sum_{z_2=1}^{z_1} \sum_{y_1=z_1+1}^{6} \sum_{y_2=z_2+1}^{y_1} \mathbb{P}\left[Y^{(1)} = y_1, Y^{(2)} = y_2\right] \mathbb{P}\left[Z^{(1)} = z_1, Z^{(2)} = z_2\right] \\
&= \frac{2890}{7776} \\
&\approx 0.372
\end{aligned}$$

Note that those events in this quadruple sum for which an argument with a subscript of 2 exceeds an argument with the same letter and subscript 1 have probability zero.

Obtain the probabilities π of the transition probability matrix P similarly using the joint distributions for $Y^{(1)}$, $Y^{(2)}$, for $Z^{(1)}$, $Z^{(2)}$, and for $W^{(1)}$, $W^{(2)}$. The exact probabilities, rounded to 3 decimal places, are in Table 5. The colors refer to the colors of the arrows representing transitions in Figure 3.

Given any initial state, the system will eventually make a transition to an absorbing state. For a transient state i, call $f_{ij}^{(n)}$ the probability that the first (and final) visit to absorbing state j occurs on the nth turn:

$$f_{ij}^{(n)} = \mathbb{P}\left[X_n = j, X_k \neq j \text{ for } k = 1, 2, \ldots, n-1 \mid X_0 = i\right].$$

Denote the $AD \times (A+D)$ matrix of these first transition probabilities by $F^{(n)}$. Then $F^{(n)} = Q^{n-1}R$. The probability that the system goes from transient state i to absorbing state j is $(I-Q)^{-1}R$. If the system ends in one of the A absorbing states then the attacker wins; if it ends in one of the D absorbing states, the defender wins. Since the initial state of a battle is the first state using the order established previously, the probability that the attacker wins is just the sum of the entries in the first row of the submatrix of the first A columns of F:

$$\mathbb{P}\left[\text{Attacker wins} \mid X_0 = (A, D)\right] = \sum_{j=1}^{A} f_{1,j}$$

and

$$\mathbb{P}\left[\text{Defender wins} \mid X_0 = (A, D)\right] = \sum_{j=A+1}^{A+D} f_{1,j}.$$

Att Dice	Def Dice	Outcome	Symbol	Exact Prob.	Approx. Prob.	Color
1	1	Def loses 1	π_{111}	15/36	0.417	light gray
1	1	Att loses 1	π_{110}	21/36	0.583	magenta
1	2	Def loses 1	π_{121}	55/216	0.254	brown
1	2	Att loses 1	π_{120}	161/216	0.745	purple
2	1	Def loses 1	π_{211}	125/216	0.579	pink
2	1	Att loses 1	π_{210}	91/216	0.421	black
2	2	Def loses 2	π_{222}	295/1296	0.228	cyan
2	2	Each lose 1	π_{221}	420/1296	0.324	darkgreen
2	2	Att loses 2	π_{220}	581/1296	0.448	yellow
3	1	Def loses 1	π_{311}	855/1296	0.660	orange
3	1	Att loses 1	π_{310}	441/1296	0.340	gray
3	2	Def loses 2	π_{322}	2890/7776	0.372	green
3	2	Each lose 1	π_{321}	2611/7776	0.336	blue
3	2	Att loses 2	π_{320}	2275/7776	0.293	red

Table 5: Probabilities in the transition probability matrix. The colors correspond to the arrows in the diagram of state transitions.

Because the state order here is the reverse of the order given in [4], the table of winning probabilities here is the reverse of Table 3 in that paper. Note also by the Markov property, that if the winning probability is calculated for say $A = 10$ and $D = 10$, then the winning probability for any war starting with $A < 10$ and $D < 10$ is also automatically calculated so that the entire Table 6 comes from the $N = (I - Q)^{-1}$ for $A = 10$ and $D = 10$.

Large engagements occur towards the end of an entire game of *Risk*. The absorbing probabilities for large battles have an interesting "saw-tooth" feature noticed in [5]. Figure 7 shows the probability mass function when the attacker and defender each start with 40 armies. The horizontal axis shows all the absorbing states of a $(40, 40)$ battle and the vertical axis is the probability of the corresponding absorbing state. The outcome states along the horizontal axis are in decreasing order of preference of the attacker. One distinctive feature of this probability mass function is the large dip in the curve. The outcomes $(1, 0)$ and $(0, 1)$ are less likely than their neighbors $(2, 0)$ and $(0, 2)$. The outcomes $(2, 0)$ and $(0, 2)$ can occur when a player with a single army will roll only one die against two dice and is less likely to

Def	10	9	8	7	6	5	4	3	2	1
Att										
10	0.568	0.480	0.380	0.287	0.193	0.118	0.057	0.021	0.001	0.000
9	0.650	0.558	0.464	0.357	0.258	0.162	0.086	0.033	0.003	0.000
8	0.724	0.646	0.547	0.446	0.329	0.224	0.123	0.054	0.005	0.000
7	0.800	0.726	0.643	0.536	0.423	0.297	0.181	0.084	0.011	0.000
6	0.861	0.808	0.730	0.640	0.521	0.397	0.253	0.134	0.021	0.000
5	0.916	0.873	0.818	0.736	0.638	0.506	0.359	0.206	0.049	0.002
4	0.954	0.930	0.888	0.834	0.745	0.638	0.477	0.315	0.091	0.007
3	0.981	0.967	0.947	0.910	0.857	0.769	0.642	0.470	0.206	0.027
2	0.994	0.990	0.980	0.967	0.934	0.890	0.785	0.656	0.363	0.106
1	1.000	1.000	1.000	0.999	0.997	0.990	0.972	0.916	0.754	0.417

Table 6: Probability that the attacker wins with A armies against D defending armies.

survive. Also interesting is the jagged or "saw-tooth" phenomenon near the local maxima. A final outcome of $(8, 0)$ or $(6, 0)$ is each more likely than an outcome of $(7, 0)$. In the largest part of the transient states the probability of removing *two* armies from one player is about double the probability of each player losing *one* army. So a path that begins at $(40, 40)$ is more likely to visit only states (a, d) in which a and d have the same parity, except when one player retains just one army.

Using the matrix NR allows calculation of the expected values and variances for the losses that the attacker and defender will suffer in a battle. For example, suppose $A = 5$ and $D = 4$, as illustrated in Figure 3. In this case, the first row of the 20×9 matrix NR gives the probabilities for the $A+D = 9$ absorbing states: $(NR)_{1,\cdot} = 0.138, 0.165, 0.179, 0.107, 0.050, 0.070, 0.124, 0.104, 0.064,$. Let R_A and R_D denote the number of armies remaining for the attacker and defender respectively given the initial state $X_0 = (A, D)$. The probability distributions for R_A and R_D are in the first row of NR:

$$\mathbb{P}\left[R_A = k\right] = \begin{cases} \sum\limits_{k-A+1}^{A+D} (NR)_{1,k} & k = 0, \\ (NR)_{1,A+1-k} & k = 1, 2, \ldots A \end{cases}$$

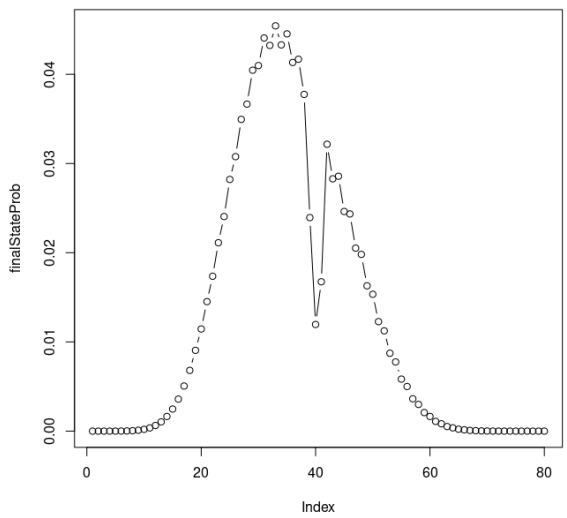

Figure 7: The probability mass function of absorbing states for $A = 40$, $D = 40$. The absorbing states are in decreasing order of preference for the attacker.

20	19	18	17	16	15	14	13	12	11
0.633	0.628	0.623	0.617	0.611	0.605	0.599	0.592	0.584	0.576
10	9	8	7	6	5	4	3	2	1
0.568	0.558	0.547	0.536	0.521	0.506	0.477	0.470	0.363	0.417

Table 7: Probability of attacker winning when the number of armies are equal.

and

$$\mathbb{P}\left[R_D = k\right] = \begin{cases} \sum_{k=1}^{A} (NR)_{1,k} & k = 0, \\ (NR)_{1,k} & k = A+1, A+2, \ldots A+D. \end{cases}$$

The mean and standard deviation for the attacker's remaining armies are $\mathbb{E}\left[R_A\right] = 2.15$ and $\sqrt{\mathrm{Var}\left[R_A\right]} = 1.89$. The defender's mean and standard deviation in this case are $\mathbb{E}\left[R_D\right] = 0.92$ and $\sqrt{\mathrm{Var}\left[R_D\right]} = 1.36$. Figure 8 has plots of expected losses for values of A and D between 1 and 20 This plot shows that the attacker has an advantage in the sense that expected losses are lower than for the defender, provided the initial number of attacking armies is not too small.

When the number of attacking and defending armies is equal, the probability that the attacker ends up winning the territory exceeds 50%, provided the initial stakes are high enough, at least 5 armies each, initially, see Table 7, With 40 armies attacking 40 defending armies, the expected armies remaining are 7.5 attacking armies and 2.1 defending armies. The overall conclusion is that the chances of winning a battle are more favorable for the attacker. The logical recommendation is then for the attacker to be aggressive.

Using the absorption probabilities for the states $(a, 0)$ where the attacker wins allows a more detailed summary of the outcomes of the war. As an example, suppose the attacker wants at least a 75 percent chance of winning the war with at least four armies surviving. With how many armies should the attacker start a war to create this favorable situation? Refer to Figure 9 for an example of how to answer this important strategic question. Each facet in the figure indicates the number of defending armies from 1 to 4. The horizontal axis in each facet lists the number of starting attacking armies, while the vertical axis in each facet displays how many attacking armies remain after the battle. Respectively, the blue, green and red curves with points indicate the first-quartile, median, and third-quartile states in

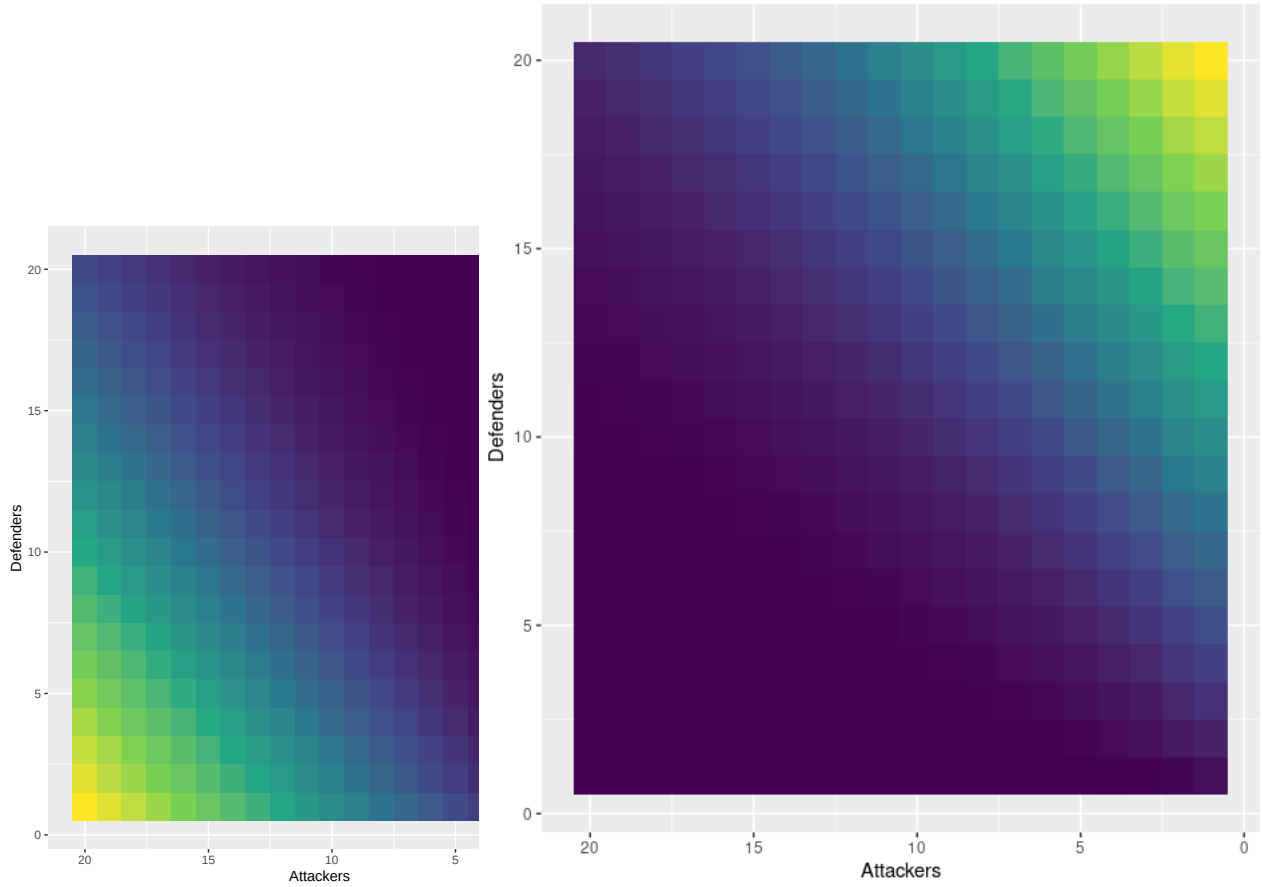

Figure 8: Expected losses for values of A and D. Note the axis scales, the number of attackers left to right, the number of attackers decrease top to bottom, as in Figure 3

the distribution of outcomes based on the number of the attacker's starting armies. The outcomes are in descending order of preference for the attacker. For example, consider the lower right facet corresponding to the defender starting with 4 armies. If the attacker has seven armies, the most desired outcome is the absorbing state $(7, 0)$, while the least desired is $(0, 4)$. In this case, the median outcome is $(4, 0)$ in the sense that $(a, 0)$ with $a \geq 4$ has a probability of at least 0.50 (in fact, $\mathbb{P}[a \geq 4, d = 0] = 0.622$.) This means that at least half of the time, the attacker will finish the war with 4 or more armies. The interquartile range is $(6, 0)$ through $(2, 0)$. This means that at least 25 percent of the time (precisely, the probability is 0.303) the attacker will win the war and have 6 or more armies remaining; at least 50 percent of the time (precisely, the probability is 0.503) the attacker will win and have 2 to 6 armies remaining; and the remaining 25 percent (precisely, the probability is 0.195) of battles will leave the attacker with fewer than 2 armies. Returning to the question of how many armies the player needs to start with in order to attack a defender with 4 armies with at least a 75 percent chance of winning with at least four armies remaining, we see that the attacker should start with at least 9 armies. In this case, the interquartile range is $(8, 0)$ through $(4, 0)$ and the median outcome is $(6, 0)$. The figure and the prior examples are illustrative, using up to 10 attacking armies and 4 defending armies. The R script below shows how to calculate these quartiles for any number of attackers and defenders.

The number of battles fought in a war is the number of dice rolls until absorption into one of the states $(a, 0)$ or $(0, d)$. This is the number of state transitions to absorption, also called the waiting time until absorption. The standard formula for calculating the expected waiting time to absorption is $(I - Q)^{-1}\mathbf{1}$. Using this, the expected number of battles for a war with 10 attacking armies against 4 defending armies is 3.994. The R script below shows how to calculate the expected waiting time to absorption for any number of attackers and defenders.

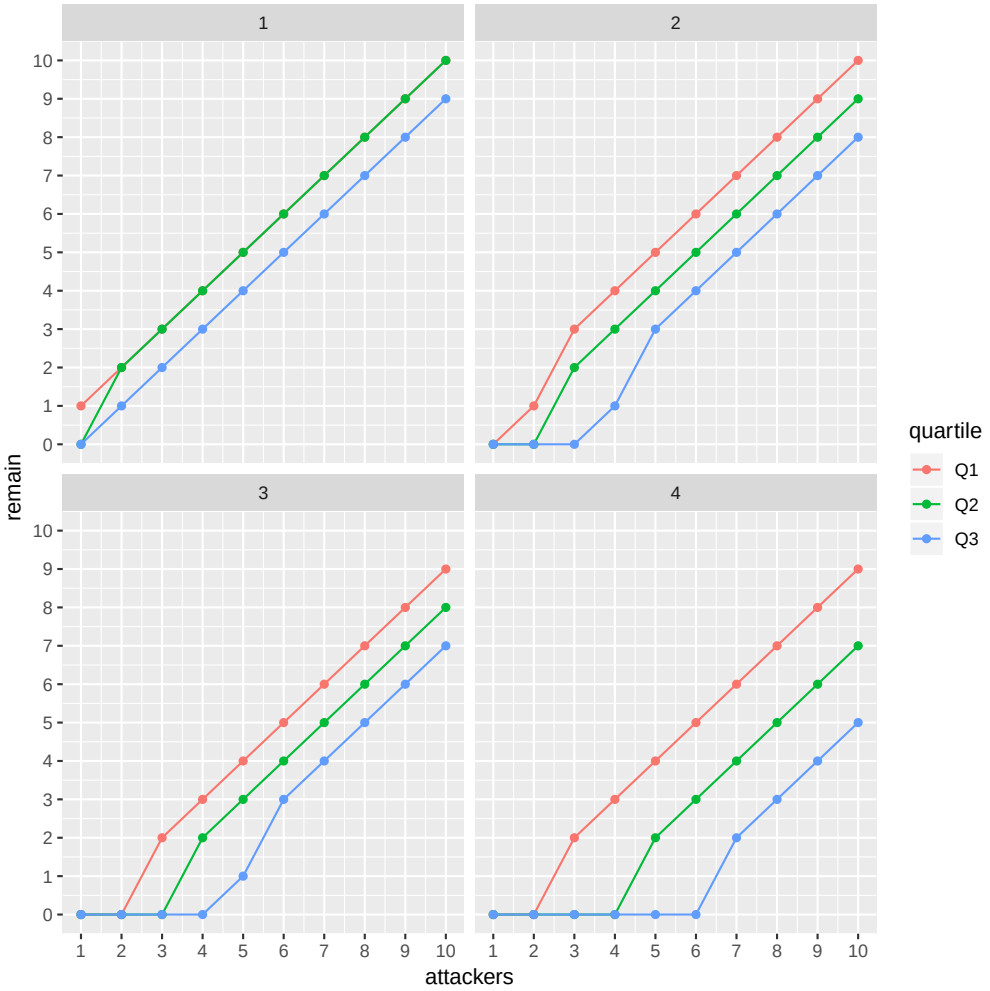

Figure 9: Quartile statistics for the number of attacking armies remaining in various scenarios.

Section Ending Answer

The future development of the game at a turn must be independent of the past to model the game as a Markov chain. Games depending on a randomizing device, such as a dice rolls or a spinner, can often be modeled as a Markov chain. Examples in this section are the games of *Tenzi*, *Chutes and Ladders*, *Count Your Chickens!* and battles from *Risk*.

Sources

The subsection on *Tenzi* is adapted from [2]. The subsection on Chutes and Ladders is adapted from [1]. The subsection on Count Your Chickens is adapted from [3]. The subsection on *Risk* is adapted from [4], [5] and [6].

Algorithms, Scripts, Simulations

Algorithm

Tenzi Game Markov Chain

```
Comment Post: Mean and standard deviation of waiting time to win
Also plots of pdf and cdf of waiting time.
Load transition probability matrix P

Iteratively compute powers of P
Add (1,11) entry to cdf list

Differences of cdf are the pdf

Check total probability and approximate mean

Extract transient probability submatrix
Compute fundamental matrix with matrix inversion

Calculate mean from fundamental matrix
Calculate variance of waiting times
Print mean and standard variation

Plot pdf and cdf with base functions.
```

Chutes and Ladders Game Markov Chain

Comment Post: Mean and standard deviation of waiting time to win
Load library "markovchain"
ROTVEC to cyclically rotate a vector
Create (almost) circulant matrix for moves according to dice roll
Create matrix for jumps due to ladders
Create matrix for jumps due to chutes
Transition matrix is product of dice roll, ladders and chutes

Create Markov chain object
Compute mean absorption time with library function

Extract transient probability submatrix
Compute fundamental matrix with matrix inversion

Calculate mean from fundamental matrix
Calculate variance of waiting times
Print mean and standard variation

Plot pdf and cdf with base functions.

Scripts

R R script for Tenzi.

```
x <- c(0.1615055828898457, 0.3230111657796915, 0.2907100492017223,
      0.1550453595742519, 0.05426587585098816, 0.01302381020423716,
      0.002170635034039527, 2.48072575318803*10^-4,
      1.860544314891023*10^-5, 8.2690858439601*10^-7,
      1.65381716879202*10^-8, 0.0, 0.1938066994678149,
      0.3488520590420667, 0.2790816472336534, 0.1302381020423716,
      0.03907143061271148, 0.007814286122542296,
      0.001041904816338973, 8.930612711476907*10^-5,
      4.465306355738455*10^-6, 9.922903012752121*10^-8, 0.0, 0.0,
      0.2325680393613778, 0.3721088629782046, 0.2604762040847432,
      0.1041904816338973, 0.02604762040847432,
```

```
    0.004167619265355891,
          4.167619265355891*10^-4, 2.381496723060509*
    10^-5,
          5.953741807651273*10^-7,0.0, 0.0, 0.0,
    0.2790816472336534,
          0.3907143061271148, 0.2344285836762689,
    0.07814286122542295,
          0.01562857224508459, 0.001875428669410151,
          1.250285779606767*10^-4, 3.572245084590763*
    10^-6,0.0, 0.0,
          0.0, 0.0, 0.3348979766803841,
    0.4018775720164609,
          0.2009387860082305, 0.05358367626886145,
    0.008037551440329218,
          6.430041152263375*10^-4, 2.143347050754458*
    10^-5,0.0, 0.0,
          0.0, 0.0, 0.0, 0.4018775720164609,
    0.4018775720164609,
          0.1607510288065844, 0.03215020576131687,
    0.003215020576131687,
          1.286008230452675*10^-4,0.0, 0.0, 0.0, 0.0, 0.0,
     0.0,
          0.4822530864197531, 0.3858024691358025,
    0.1157407407407407,
          0.0154320987654321, 7.716049382716049*10^-4,0.0,
     0.0, 0.0,
          0.0, 0.0, 0.0, 0.0, 0.5787037037037037,
    0.3472222222222222,
          0.06944444444444445, 0.004629629629629629,0.0,
    0.0, 0.0, 0.0,
          0.0, 0.0, 0.0, 0.0, 0.6944444444444444,
    0.2777777777777778,
          0.02777777777777778,0.0, 0.0, 0.0, 0.0, 0.0,
    0.0, 0.0, 0.0,
          0.0, 0.8333333333333334, 0.1666666666666667,0.0,
     0.0, 0.0,
          0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0)
P  <-  t( matrix(x, 11, 11) )

PPow  <- P
tenzicdf  <- c(PPow[1,11])
for (i in 2:50) {
  PPow  <- PPow %*% P;
  tenzicdf  <- c(tenzicdf, PPow[1,11])
}
```

```r

tenzipdf  <-  tenzicdf[2:50] - tenzicdf[1:49]
tenzipdf  <- c(tenzicdf[1], tenzipdf)

sum(tenzipdf)
tenzimean  <- (1:50) %*% tenzipdf

Q  <-  P[1:10, 1:10]
I10  <-  diag(10)
N  <- solve( I10 - Q, I10)

mean  <- N %*% rep(1,10)
Varw  <- ((2 * N - I10) %*% N) %*% rep(1, 10) - ( N %*% rep(1, 10) )^2
sdw  <- sqrt(Varw)
cat("Mean waiting time: ", mean[1], "\nSD of waiting time:", sdw[1], "\n");

plot(x = 1:50, tenzicdf, col="red", xlab="Rolls", ylab="Probability")
points(1:50, tenzicdf, type="l", col="red")
points(1:50, tenzipdf, col="blue")
points(1:50, tenzipdf, type="l", col="blue")
```

R R script for Chutes and Ladders.

```r
library("markovchain")

rotvec <- function(vec) vec[ c(length(vec), 1:(length(vec)-1)) ]

nStates <- 100
chutesladdersStates <- as.character( c(1:nStates) )

diceRoll <- matrix(0, nStates, nStates)
p1 <- c(0,
        1/6, 1/6, 1/6, 1/6, 1/6, 1/6,
        numeric(nStates-7)
        )
for (i in 1:(nStates-5)) {
    diceRoll[i, ] <- p1
    p1 <- rotvec(p1)
}
diceRoll[100, 100] <- 1
```

```r

diceRoll[99, 99] <- 5/6; diceRoll[99, 100] <- 1/6

diceRoll[98, 98] <- 4/6; diceRoll[98, 99] <- 1/6; diceRoll[98, 100] <- 1/6

diceRoll[97, 97] <- 3/6; diceRoll[97, 98] <- 1/6
diceRoll[97, 99] <- 1/6; diceRoll[97, 100] <- 1/6

diceRoll[96, 96] <- 2/6; diceRoll[96, 97] <- 1/6
diceRoll[96, 98] <- 1/6; diceRoll[96, 99] <- 1/6; diceRoll[96, 100] <- 1/6

ladders <- diag(nStates)
ladderIn <-  c( 1,  4,  9, 21, 28, 36, 51, 71,  80)
ladderOut <- c(38, 14, 31, 42, 84, 44, 67, 91, 100)
ladders[ cbind(ladderIn, ladderIn) ] <- 0
ladders[ cbind(ladderIn, ladderOut) ] <- 1

chutes <- diag(nStates)
chuteIn <-  c(16, 47, 49, 56, 62, 64, 87, 93, 95, 98)
chuteOut <- c( 6, 26, 11, 53, 19, 60, 24, 73, 75, 78)
chutes[ cbind(chuteIn, chuteIn) ] <- 0
chutes[ cbind(chuteIn, chuteOut) ] <- 1

chutesladdersMat <-  diceRoll %*% ladders %*% chutes

chutesladders <- new("markovchain", states = chutesladdersStates, byrow = TRUE,
                     transitionMatrix = chutesladdersMat, name = "ChutesLadders")
meanMC  <- meanAbsorptionTime(object=chutesladders)

Q  <- chutesladdersMat[1:99, 1:99]
I99  <-  diag(99)
N  <- solve( I99 - Q, I99)
absorptionTimeMat  <-  N %*% rep(1, 99)
meanMat  <- mean( absorptionTimeMat[ c(38, 2, 3, 14, 5, 6)] )
varabsorptionTimeMat  <- ((2 * N - I99) %*% N) %*% rep(1, 99) - ( N %*% rep(1, 99) )^2
sdAbsorptionTime  <-  sqrt( mean( varWait[ c(38, 2, 3, 14, 5, 6) ] ) )

cat("Mean waiting time from markovchain library: ",
```

```
    meanMC[1],
"\nMean waiting time from normal matrix: ", meanMat + 1,
"\nSD of waiting time:", sdAbsorptionTime[1], "\n");
```

R R script for *Count Your Chickens!*.

```

boardLength <- 40

animalList <- c("C", "D", "P", "S", "T")
NAnimals <- length(animalList) + 1  # one more for fox

blueSquares <- numeric(boardLength)
blueSquares[4] <- 1
blueSquares[8] <- 1
blueSquares[22] <- 1
blueSquares[35] <- 1
blueSquares[39] <- 1

totalNumberOfBlues <- sum(blueSquares)

# Setting up the board
board <- character(boardLength)

board[1] <- "B"
board[2] <- "S"
board[3] <- "P"
board[4] <- "T"
board[5] <- "C"
board[6] <- "D"
board[7] <- "P"
board[8] <- "C"
board[9] <- "D"
board[10] <- "S"
board[11] <- "T"
board[12] <- "B"
board[13] <- "C"
board[14] <- "P"
board[15] <- "B"
board[16] <- "B"
board[17] <- "B"
board[18] <- "T"
board[19] <- "B"
board[20] <- "T"
```

```r
board[21] <- "D"
board[22] <- "S"
board[23] <- "C"
board[24] <- "D"
board[25] <- "P"
board[26] <- "T"
board[27] <- "B"
board[28] <- "S"
board[29] <- "C"
board[30] <- "B"
board[31] <- "B"
board[32] <- "T"
board[33] <- "P"
board[34] <- "S"
board[35] <- "D"
board[36] <- "B"
board[37] <- "S"
board[38] <- "C"
board[39] <- "P"
board[40] <- "CDPST"

# Count how many states we have
statesIndex <- 1  #initial state (0,0)
statesCount <- 1
statesHolder <- matrix(0, boardLength * totalNumberOfBlues, 2)

for (i in 1:(boardLength - 1)) {
    # don't use last space, the 'coop'
    if (board[i] != "B") {
        bLi <- sum(blueSquares[1:i])  #blueSquares to Left of i, including i
        bRi <- sum(blueSquares[(i + 1):boardLength])  #blueSquares to Right of i

        maxChickens <- min(boardLength, i + bLi)
        minChickens <- max(0, i - bRi)

        statesCount <- statesCount + maxChickens - minChickens + 1
        for (j in minChickens:maxChickens) {
            statesIndex <- statesIndex + 1
            statesHolder[statesIndex, ] <- c(i, j)
        }
    }
```

```
}
statesHolder[statesCount + 1, ] <- c(Inf, Inf)  # win state
statesHolder[statesCount + 2, ] <- c(-Inf, -Inf)  # lose state
statesCount <- statesCount + 2  #add final won and loss states
states <- statesHolder[1:statesCount, ]
rm(statesHolder)

Q <- matrix(0, statesCount - 2, statesCount - 2)
R <- matrix(0, statesCount - 2, 2)
I <- diag(2)
Z <- matrix(0, 2, statesCount - 2)

for (i in 1:(statesCount - 2)) {
    currentSquare <- states[i, 1]
    chicksInCoop <- states[i, 2]
    currentState <- i

    # This sets transition probabilities from spinning a fox
    if (chicksInCoop == 0) {
        Q[i, i] <- 1/NAnimals  # stay in current state, prob 1/NAnimals
    } else if (states[i - 1, 1] == currentSquare) {
        Q[i, i - 1] <- 1/NAnimals  # remove 1 chick, fill trans prob
    } else {
        # too few chicks, must lose
        R[i, 2] <- R[i, 2] + 1/NAnimals  # so add to lose probability
    }

    # Now set transition probabilities from other five animals.
    for (animal in animalList) {

        nextSquare <- grep(animal, board[(currentSquare + 1):boardLength])[1] + currentSquare

        if (blueSquares[nextSquare] == 0) {
            # not Blue, just add advance
            updatedChicksInCoop <- chicksInCoop + (nextSquare - currentSquare)
```

```
            if (updatedChicksInCoop > boardLength)
                updatedChicksInCoop <- boardLength
        } else {
            # Blue square, so add extra chick
            updatedChicksInCoop <- chicksInCoop + (nextSquare - currentSquare) +
                1
            if (updatedChicksInCoop > boardLength)
                updatedChicksInCoop <- boardLength
        }

        in_states <- FALSE

        for (p in 1:statesCount) {
            if (states[p, 1] == nextSquare && states[p, 2] == updatedChicksInCoop) {
                in_states <- TRUE
                newLocation <- p
                break
            }
        }

        if (in_states == TRUE) {
            Q[i, newLocation] <- 1/NAnimals
        } else {
            if (nextSquare == boardLength && updatedChicksInCoop == boardLength)
                {
                  R[i, 1] <- R[i, 1] + 1/NAnimals
                }  # win
 else {
                R[i, 2] <- R[i, 2] + 1/NAnimals  #lose
            }
        }
    }
}

# Calculate win-lose probabilities with normal matrix: (I-Q)^{-1} R
IsC <- diag(statesCount - 2)
matrixProbs <- solve(IsC - Q, R)
```

R R script for *Risk*.

```r
A <- 10
D  <- 10

totalStates <- A * D + (A + D)

stateToIndex <- function(a,d) {
    # given a state, retrun the index for the trans prob matrix
    if (a > 0) {
        ind <- (A+1 - a) + (D-d)*A
    }  else {
        ind <- A*D + A + d
    }
    ind
}

indexToState <- function(ind) {
    # given an index for a transient state, return the state as a pair
    if (ind <= A*D + A) {
        j <- A*D - ind
        r <- j %% A
        first <- 1 + r
        second <- 1 + (j-r)/A
    } else {
        first <- 0
        second <- ind - (A*D + A)
    }
    c(first, second)
}

pi111 <- 15/36                          # light gray
pi110 <- 21/36                          # magenta

pi121 <- 55/216                         # brown
pi120 <- 161/216                        # purple

pi211 <- 125/216                        # pink
pi210 <- 91/216                         # black

pi222 <- 295/1296                       # cyan
pi221 <- 420/1296                       # darkgreen
pi220 <- 581/1296                       # yellow

pi311 <- 855/1296                       # orange
```

```r
pi310 <- 441/1296                          # gray

pi322 <- 2890/7776                         # green
pi321 <- 2611/7776                         # blue
pi320 <- 2275/7776                         # red

QR <- matrix(0, A * D, A * D + A + D)
## R <- matrix(0, A * D, A + D)
I <- diag(A + D)
Z <- matrix(0, A + D, A * D)

att3dice <- seq(from=A, to = 3, by=-1)
def2dice <- seq(from=D, to = 2, by=-1)

for (d in def2dice) {
    for (a in att3dice) {
        QR[stateToIndex(a,d), stateToIndex(a, d-2)] <- pi322   # green
        QR[stateToIndex(a,d), stateToIndex(a-1, d-1)] <- pi321 # blue
        QR[stateToIndex(a,d), stateToIndex(a-2, d)] <- pi320 # red
    }
    QR[stateToIndex(2, d), stateToIndex(2, d-2)] <- pi222 # cyan
    QR[stateToIndex(2, d), stateToIndex(1, d-1)] <- pi221 # darkgreen
    QR[stateToIndex(2, d), stateToIndex(0, d)] <- pi220 # yellow

    QR[stateToIndex(1, d), stateToIndex(1, d-1)] <- pi121 # purple
    QR[stateToIndex(1, d), stateToIndex(0, d)] <- pi120 # brown
}

for (a in att3dice) {
    QR[stateToIndex(a, 1), stateToIndex(a, 0)] <- pi311 # orange
    QR[stateToIndex(a, 1), stateToIndex(a-1, 1)] <- pi310 # gray
}

QR[stateToIndex(2, 1), stateToIndex(2, 0)] <- pi211 # pink
```

```r
QR[stateToIndex(2, 1), stateToIndex(1, 1)] <- pi210 # black
QR[stateToIndex(1, 1), stateToIndex(1, 0)] <- pi111 # lightgray
QR[stateToIndex(1, 1), stateToIndex(0, 1)] <- pi110 # magenta


M <- rbind(QR, cbind(Z, I))

library("markovchain")

riskChain <- new("markovchain", states = as.character(1:totalStates), byrow = TRUE,
    transitionMatrix = M, name = "RISK")
absorpProb <- absorptionProbabilities(riskChain)
attackerWins  <- apply(absorpProb[ , 1:A], 1, sum)
attackerWinTable  <- t(matrix(formatC(attackerWins, digits=3, format="f"), A,D))
attackerWinTable
```

Problems to Work for Understanding

1: The CDF and PDF functions $P(k)$ and $p(k)$ above respectively summarize the probability that a single player will get Tenzi in k rolls. What if n people are playing? This essentially taking n independent random samples from the probability distribution $p(k)$, and identifying the minimum value(s) of k as the roll in which the game is won. For example, if there are $n = 5$ players, we take 5 independent random samples from $p(k)$. Suppose the arbitrary values of k are $[12, 20, 15, 9, 28]$, representing the number of rolls needed for each player to get Tenzi. Then the 4th player wins the game in 9 rolls. In general, write an expression for the probability that the minimum value from

n random samples of $p(k)$ is less than or equal to k in terms of $P(k)$.

2: Write a script in some computer language to replicate the calculations and graphs for the Markov chain for *Tenzi*.

3: Show that the shortest possible game of *Chutes and Ladders* consists of 7 moves by listing those moves.

4: Consider the effect of chutes and ladders on the waiting time to absorption.

(a) Find the expected waiting time to absorption in a game with no chutes or ladders.

(b) Insert a ladder from state 46 to state 94. What happens to the expected waiting time?

(c) Instead insert a chute from state 83 to state 7. What happens to the expected waiting time?

(d) Add a ladder to the original game from state 79 to state 81. What happens to the expected waiting time?

(e) Add a chute to the original game from state 29 to state 27. What happens to the expected waiting time?

5: Investigate the following modifications to the small *Count Your Chickens!* game.

(a) What is the win probability if only square 2 is blue? What is the win probability if squares 2, 3 and 5 are blue?

(b) What effect does adding additional animals to the board (i.e., increasing the size of the spinner) have?

(c) What if the fox removes more than one chick from the coop?

(d) What effect does changing the size of each character's space in the spinner do? For example, what if the fox covered 1/3 of the spinner and the remaining five animals each took up 2/15 of the spinner?

(e) What if some of the blue squares were colored green and had the effect of moving an additional two chicks into the coop?

(f) Create your own questions that can be analyzed using Markov chains and to modify the supplemental R code to answer the questions.

6: For the game of *Risk*, show that the probability of the attacker rolling three dice, the defender rolling one die and the defender losing one army is 855/1296. That is, calculate π_{310}, the gray arrow in Figure 3 and Table 5.

7: Find the expected number of attacking armies remaining when attacking an equal number of defending armies, for $n = 20$ to $n = 1$.

Reading Suggestion:

References

[1] S. C. Althoen, L. King, and K. Schilling. How long is a game of snakes and ladders. *The Mathematical Gazette*, 77(478):71–76, March 1993.

[2] Steve Bacinski and Timothy Pennings. How to win at tenzi. *The College Mathematics Journal*, 51(4):242–253, 2020.

[3] David McCune and Lori McCune. Counting your chickens with markov chains. *Mathematics Magazine*, 92(3):163–172, 2019.

[4] Jason A. Osborne. Markov chains for the risk board game revisited. *Mathematics Magazine*, 76(2):129–135, April 2003.

[5] Pamela Pierce and Robert Wooster. Conquer the world with markov chains. *Math Horizons*, 22(4):18–21, April 2015 2015.

[6] Bariş Tan. Markov chains and the risk board game. *Mathematics Magazine*, 70(5):349–357, 1997.

Outside Readings and Links:

1.

2.

3.

4.

Solutions

1: Denote as $P(n,k)$ the probability that the minimum value from n random samples of $p(k)$ is less than or equal to k. Then $P(n,k)$ is the complement of the probability that all n random samples (which are independent of each other) are greater than k, so $P(n,k) = 1 - (1 - P(k))^n$. The pdf giving the probability that a game with n players is won in exactly k rolls is $p(n,k) = P(n,k) - P(n,k-1)$.

2: The transition probability matrix for *Tenzi* (rounded to three decimal places) is written in Maxima as

```
pi[i,j] := if (j >= i) then  binomial(11-i, j-i) * ((1/6)^(j-i)) *
((5/6)^(11-j)) else 0;
P : genmatrix(pi, 11, 11);
fpprintprec : 3;
float(P);
```

Transfer this matrix of values to R and evaluate with the following script:

```
x  <-  c(0.1615055828898457, 0.3230111657796915, 0.2907100492017223,
         0.1550453595742519, 0.05426587585098816, 0.01302381020423716,
         0.002170635034039527, 2.48072575318803*10^-4,
         1.860544314891023*10^-5, 8.2690858439601*10^-7,
         1.65381716879202*10^-8, 0.0, 0.1938066994678149,
         0.3488520590420667, 0.2790816472336534, 0.1302381020423716,
         0.03907143061271148, 0.007814286122542296,
         0.001041904816338973, 8.930612711476907*10^-5,
         4.465306355738455*10^-6, 9.922903012752121*10^-8, 0.0, 0.0,
         0.2325680393613778, 0.3721088629782046, 0.2604762040847432,
```

```
         0.1041904816338973, 0.02604762040847432, 0.004167619265355891,
         4.167619265355891*10^-4, 2.381496723060509*10^-5,
         5.953741807651273*10^-7,0.0, 0.0, 0.0, 0.2790816472336534,
         0.3907143061271148, 0.2344285836762689, 0.07814286122542295,
         0.01562857224508459, 0.001875428669410151,
         1.250285779606767*10^-4, 3.572245084590763*10^-6,0.0, 0.0,
         0.0, 0.0, 0.3348979766803841, 0.4018775720164609,
         0.2009387860082305, 0.05358367626886145, 0.008037551440329218,
         6.430041152263375*10^-4, 2.143347050754458*10^-5,0.0, 0.0,
         0.0, 0.0, 0.0, 0.4018775720164609, 0.4018775720164609,
         0.1607510288065844, 0.03215020576131687, 0.003215020576131687,
         1.286008230452675*10^-4,0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
         0.4822530864197531, 0.3858024691358025, 0.1157407407407407,
         0.0154320987654321, 7.716049382716049*10^-4,0.0, 0.0, 0.0,
         0.0, 0.0, 0.0, 0.0, 0.5787037037037037, 0.3472222222222222,
         0.06944444444444445, 0.004629629629629629,0.0, 0.0, 0.0, 0.0,
         0.0, 0.0, 0.0, 0.0, 0.6944444444444444, 0.2777777777777778,
         0.02777777777777778,0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
         0.0, 0.8333333333333334, 0.1666666666666667,0.0, 0.0, 0.0,
         0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0)
P  <-  t( matrix(x, 11, 11) )

PPow  <- P
tenzicdf  <- c(PPow[1,11])
for (i in 2:50) {
  PPow  <- PPow %*% P;
  tenzicdf  <- c(tenzicdf, PPow[1,11])
}

library(ggplot2)
ggplot(data = as.data.frame(tenzicdf), aes(x=1:50, y=tenzicdf))
  + geom_point()
  + geom_line()

tenzipdf  <-  tenzipdf[2:50] - tenzipdf[1:49]
tenzipdf  <- c(tenzicdf[1], tenzipdf)
ggplot(data = as.data.frame(tenzipdf), aes(x=1:50, y=tenzipdf))
    + geom_point()
```

```
    + geom_line()

sum(tenzipdf)
tenzimean  <- (1:50) %*% tenzipdf

Q  <-  P[1:10, 1:10]
I10  <-  ident(10)
N  <- solve( I10 - Q, I10)
N %*% rep(1,10)

Varw  <- ((2 * N - I10) %*% N) %*% rep(1, 10) - ( N %*% rep(1, 10) )^2
sdw  <- sqrt(Varw)
```

3: A possible path is to roll a 1, to 38; roll a 6, to 44; roll a 6, to 50; roll a 1, to 67; roll a 6, to 73; roll a 6, to 79 and finally, roll a 1 to ascend from 80 to 100. Examination of the first 6 powers of the transition probability matrix show that the probability of 6 step paths from 38 is 0.007265947 but 0 for fewer than 6 steps.

Another possible path is roll a 4, to 14; roll a 6, to 20; roll a 6, to 26; roll a 2, to 84; roll a 6, to 90; roll a 5, to 75 and finally, roll a 5 to ascend from 80 to 100. Examination of the first 6 powers of the transition probability matrix show that the probability of 6 step paths from 14 is 0.001800412 but 0 for fewer than 6 steps.

4:

(a) The expected length of a game with no chutes or ladders is 35.68571. Here

(b) The expected length of the game drops from about 39.2 to about 29.8;

(c) The game lengthens to about 45.8 moves.

(d) Lengthens the expected playing time by more than two moves to about 41.9, since it increases the chances of missing the important ladder from state 80 to 100.

(e) Shortens the game by over a move to about 38.0, since it gives a second chance at the long ladder from 28 to 84.

5: The win probability if only square 2 is blue s 0.5288 The win probability if squares 2, 3 and 5 are blue is $0,7761$. The remaining questions make a good project for student investigation.

6: Use the order statistic probability $Y^{(1)} = (3y^2 - 3y + 1)/216$. The possibilities for the defender to lose one army are the following:

(a) The largest die of the 3 rolled by the attacker is a 2, and the value rolled by the defender is 1 which occurs with probability $\frac{7}{216} \cdot \frac{1}{6} = \frac{7}{1296}$.

(b) The largest die of the 3 rolled by the attacker is a 3, and the value rolled by the defender is 1 or 2 which occurs with probability $\frac{19}{216} \cdot \frac{2}{6} = \frac{38}{1296}$ The largest die of the 3 rolled by the attacker is a 4, and the value rolled by the defender is 1 or 2 or 3. which occurs with probability $\frac{37}{216} \cdot \frac{3}{6} = \frac{111}{1296}$ The largest die of the 3 rolled by the attacker is a 5, and the value rolled by the defender is 1, 2, 3, or 4 which occurs with probability $\frac{61}{216} \cdot \frac{4}{6} = \frac{244}{1296}$ The largest die of the 3 rolled by the attacker is a 6, and the value rolled by the defender is 1, 2, 3, 4, or 5 which occurs with probability $\frac{91}{216} \cdot \frac{5}{6} = \frac{455}{1296}$ The total probability is $(7+38+111+244+455)/1296 = 855/1296$.

7:

20	19	18	17	16
4.441	4.277	4.111	3.943	3.773
15	14	13	12	11
3.601	3.427	3.249	3.068	2.883
10	9	8	7	6
2.692	2.498	2.293	2.085	1.854
5	4	3	2	1
1.631	1.335	1.110	0.590	0.417

I check all the information on each page for correctness and typographical errors. Nevertheless, some errors may occur and I would be grateful if you would alert me to such errors. I make every reasonable effort to present current and accurate information for public use, however I do not guarantee the accuracy or timeliness of information on this website. Your use of the information from this website is strictly voluntary and at your risk.

I have checked the links to external sites for usefulness. Links to external websites are provided as a convenience. I do not endorse, control, monitor, or guarantee the information contained in any external website. I don't guarantee that the links are active at all times. Use the links here with the same caution as

you would all information on the Internet. This website reflects the thoughts, interests and opinions of its author. They do not explicitly represent official positions or policies of my employer.

Information on this website is subject to change without notice.

Steve Dunbar's Home Page, `http://www.math.unl.edu/~sdunbar1`

Email to Steve Dunbar, `sdunbar1 at unl dot edu`

Last modified: Processed from LaTeX source on November 13, 2020